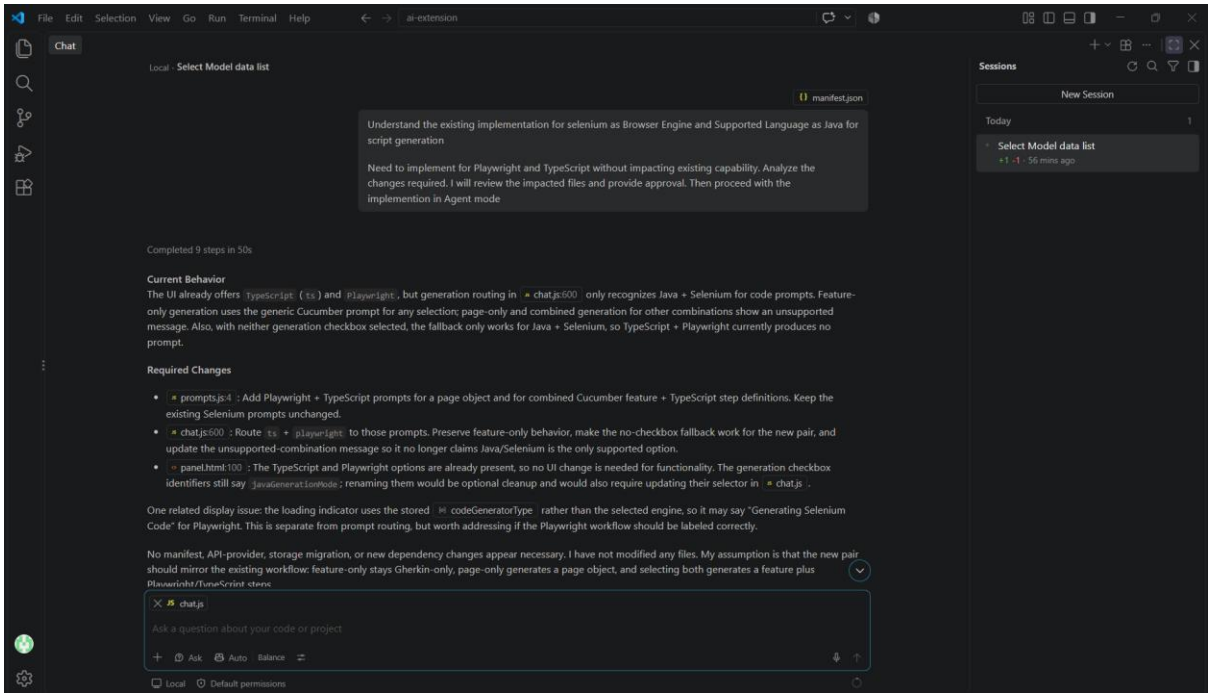
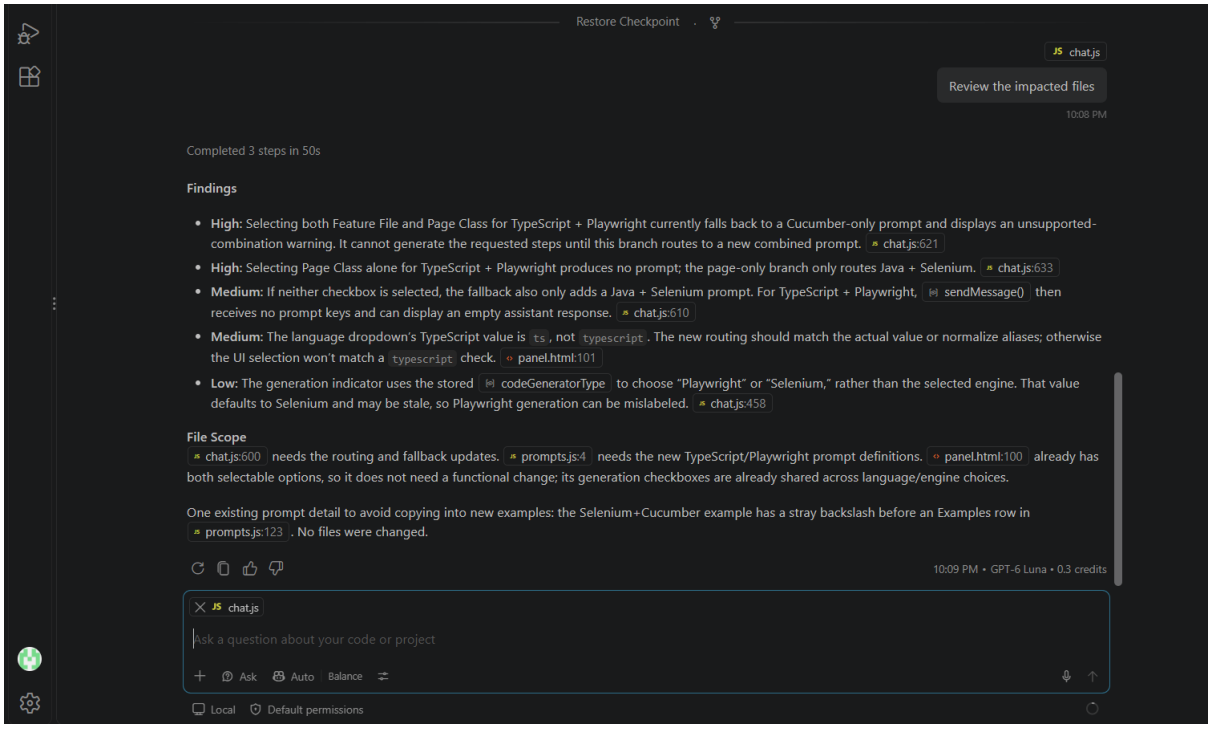
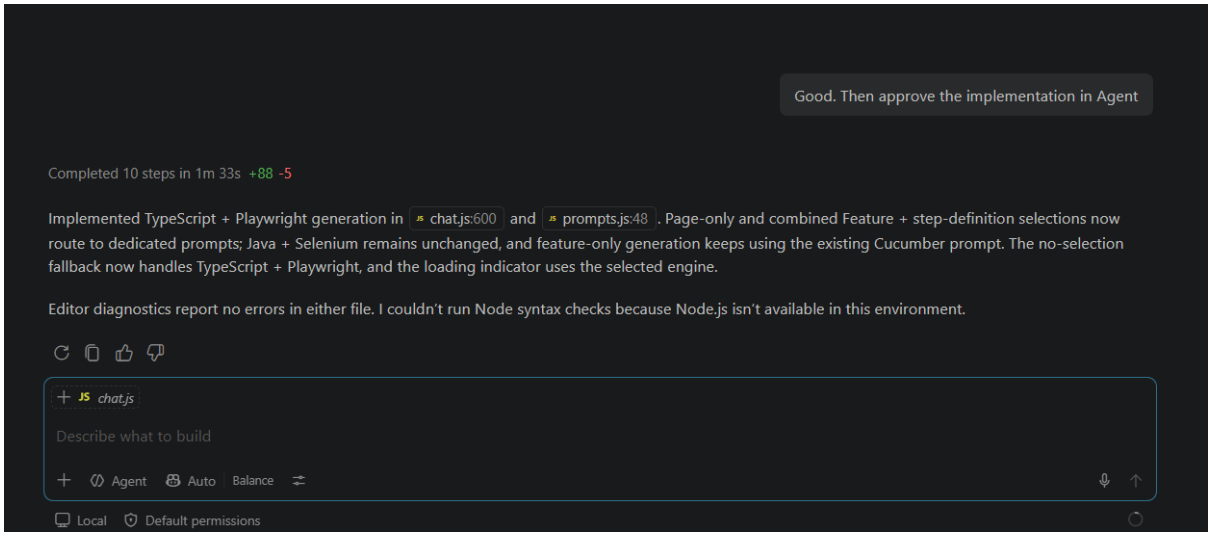
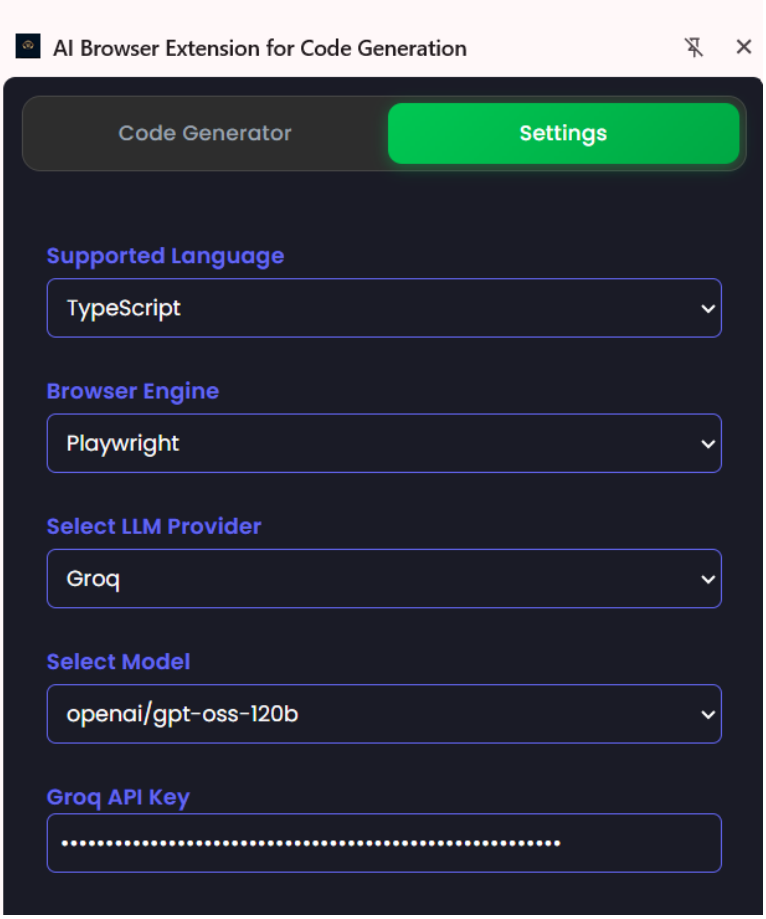
AI Extension – Implementation of Playwright with TypeScript & Selenium with C#

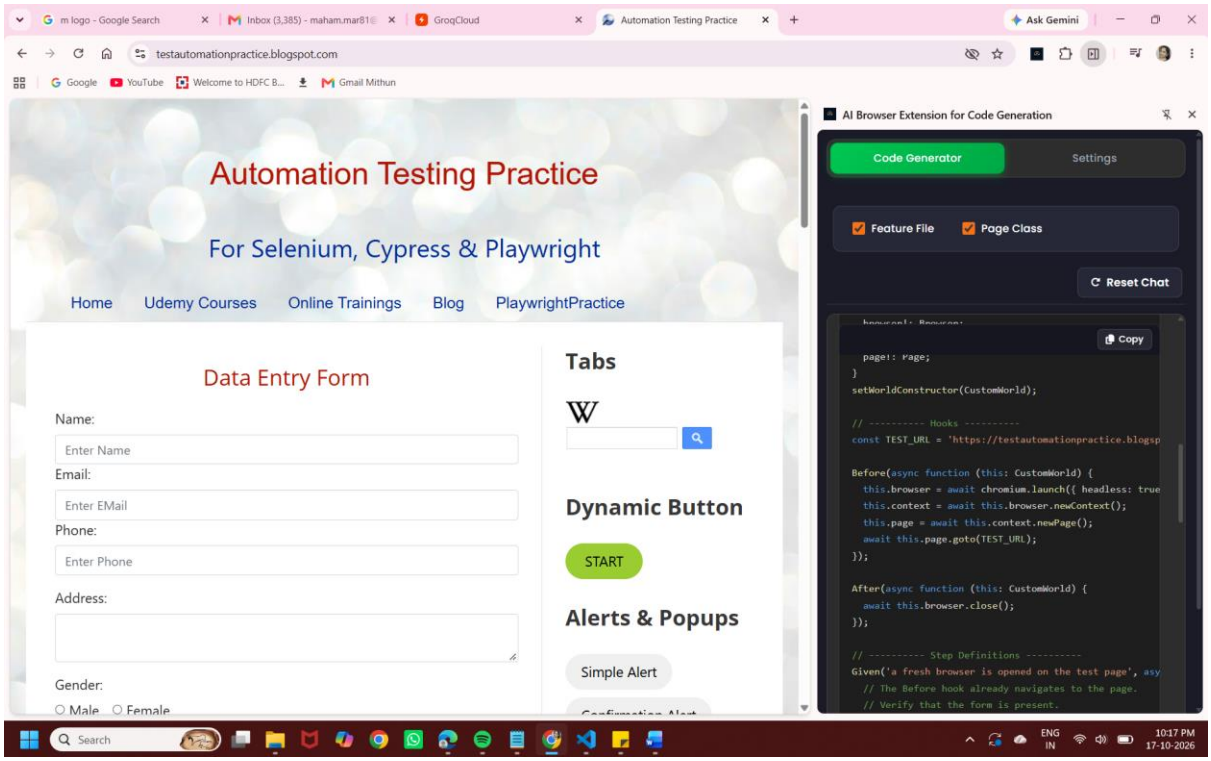
- **Playwright with TypeScript**






Good. Then approve the implementation in Agent
Completed 10 steps in 1m 33s +88 -5
Implemented TypeScript + Playwright generation in chat.js:600 and prompts.js:48. Page-only and combined Feature + step-definition selections now route to dedicated prompts; Java + Selenium remains unchanged, and feature-only generation keeps using the existing Cucumber prompt. The no-selection fallback now handles TypeScript + Playwright, and the loading indicator uses the selected engine.
Editor diagnostics report no errors in either file. I couldn't run Node syntax checks because Node.js isn't available in this environment.
chat.js
Describe what to build
Agent
Auto
Balance
Local
Default permissions


AI Browser Extension for Code Generation
Code Generator
Settings
Supported Language
TypeScript
Browser Engine
Playwright
Select LLM Provider
Groq
Select Model
openai/gpt-oss-120b
Groq API Key




```
Feature: Data Entry Form automation

  Verify that the data entry form can be filled, submitted and the date-range result is displayed correctly.

  Background:
    Given a fresh browser is opened on the test page

  Scenario Outline: Fill the form with various data sets and validate the result
    When I fill the text fields with name "<name>", email "<email>", phone "<phone>" and address "<address>"
    And I select gender "<gender>"
    And I select the following days "<days>"
    And I choose country "<country>"
    And I select colors "<colors>"
    And I select animals "<animals>"
    And I set the date range from "<startDate>" to "<endDate>"
    And I submit the form
    Then I should see the result message "<expectedResult>"

    Examples:
      | name    | email               | phone      | address          | gender | days                    | country | colors      | animals         | startDate  | endDate    |
expectedResult                                                
```

```
import { setWorldConstructor, World, Before, After, Given, When, Then } from '@cucumber/cucumber';
import { chromium, Browser, BrowserContext, Page } from 'playwright';
import assert from 'assert';

// ---------- Custom World ----------
class CustomWorld extends World {
  browser!: Browser;
  context!: BrowserContext;
  page!: Page;
}
setWorldConstructor(CustomWorld);

// ---------- Hooks ----------
const TEST_URL = 'https://testautomationpractice.blogspot.com/';

Before(async function (this: CustomWorld) {
  this.browser = await chromium.launch({ headless: true });
  this.context = await this.browser.newContext();
  this.page = await this.context.newPage();
  await this.page.goto(TEST_URL);
});

After(async function (this: CustomWorld) {
  await this.browser.close();
});

// ---------- Step Definitions ----------
Given('a fresh browser is opened on the test page', async function (this: CustomWorld) {
  // The Before hook already navigates to the page.
  // Verify that the form is present.
  await this.page.waitForSelector('#name');
});

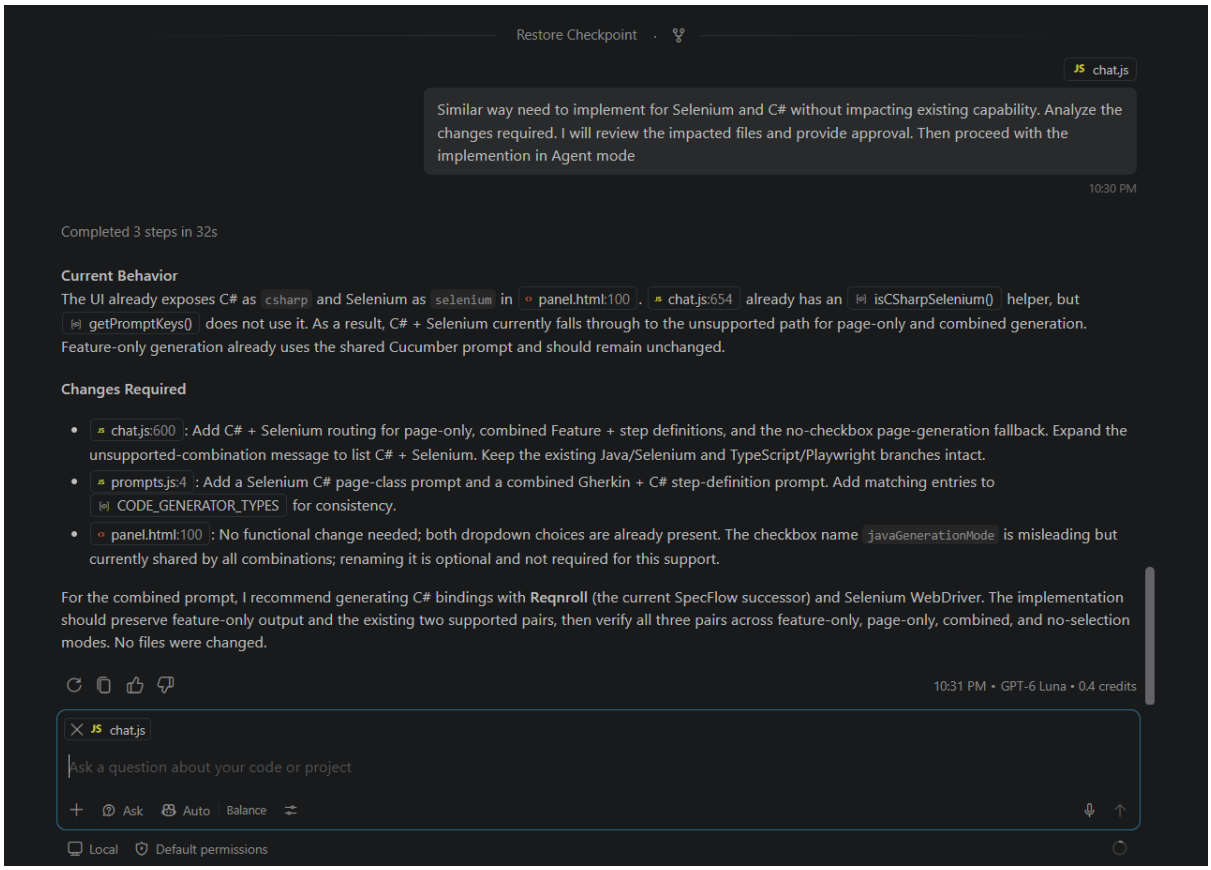
When(
  'I fill the text fields with name {string}, email {string}, phone {string} and address {string}',
  async function (this: CustomWorld, name: string, email: string, phone: string, address: string) {
    await this.page.fill('#name', name);
    await this.page.fill('#email', email);
    await this.page.fill('#phone', phone);
    await this.page.fill('#textarea', address);
  }
);

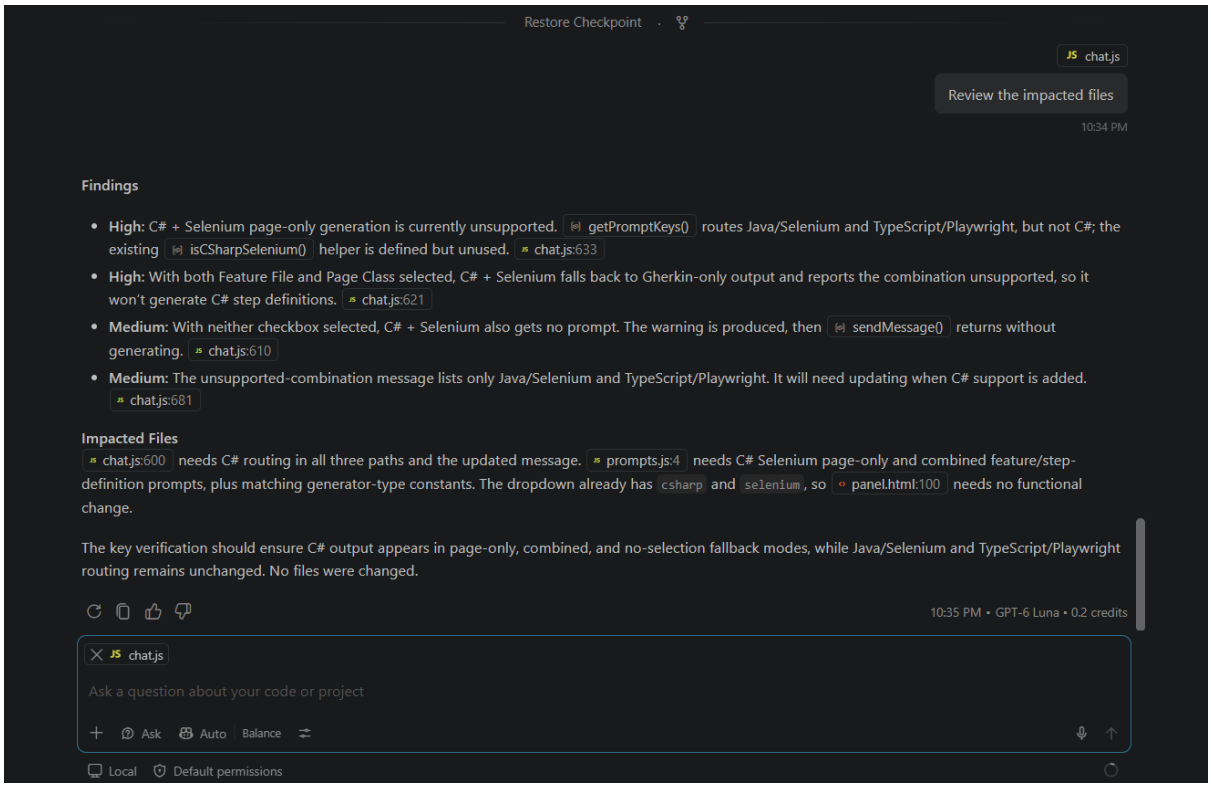
When('I select gender {string}', async function (this: CustomWorld, gender: string) {
  const genderLower = gender.toLowerCase();
  const selector = genderLower === 'male' ? '#male' : '#female';
  await this.page.check(selector);
});

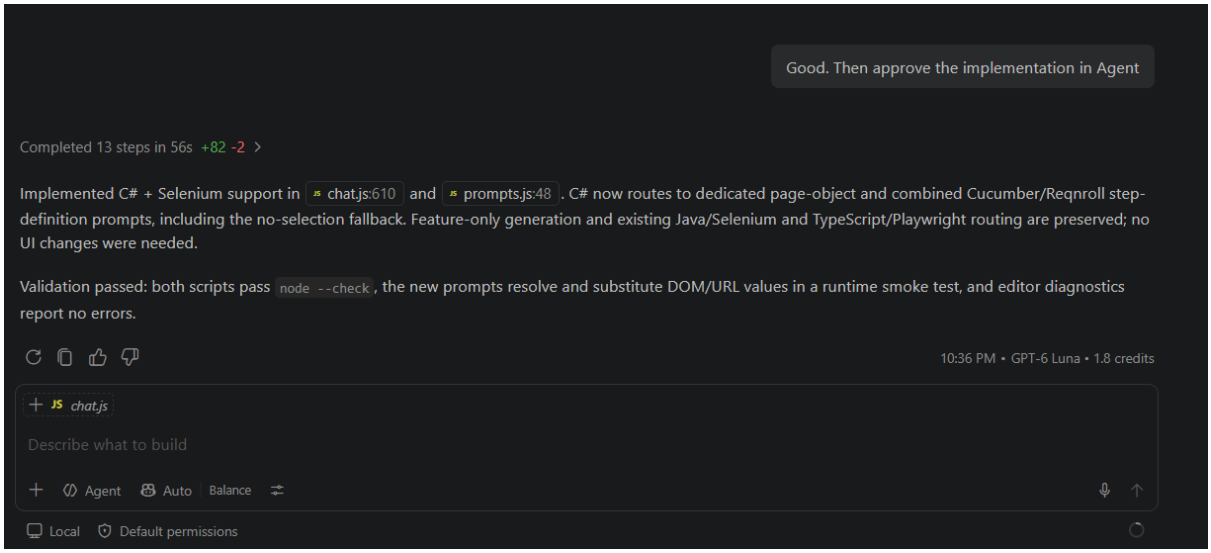
When('I select the following days {string}', async function (this: CustomWorld, days: string) {
  const dayList = days.split(',').map(d => d.trim().toLowerCase());
  for (const day of dayList) {
    const selector = `#${day}`;
    await this.page.check(selector);
  }
});

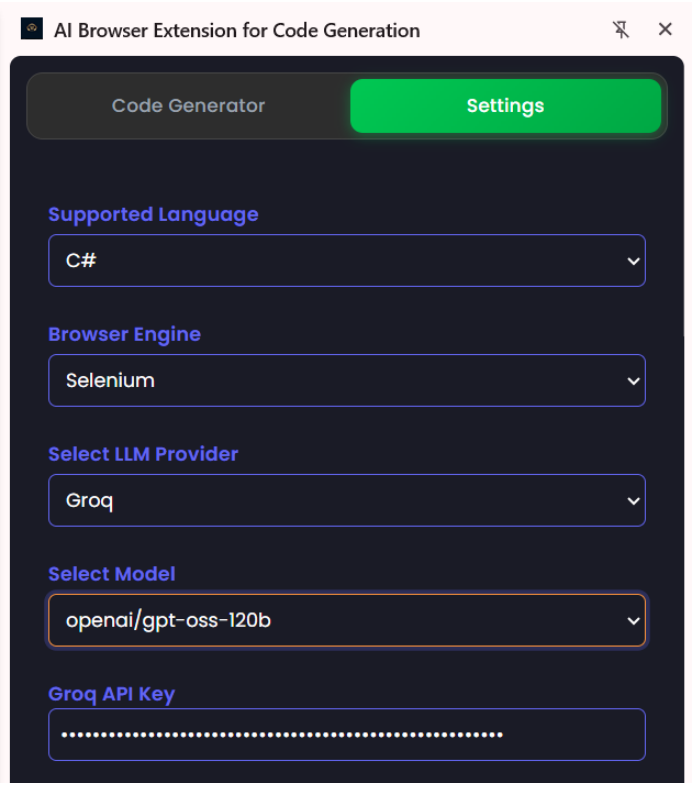
When('I choose country {string}', async function (this: CustomWorld, country: string) {
  const valueMap: Record<string, string> = {
    usa: 'usa',
    canada: 'canada',
    uk: 'uk',
  };
  await this.page.selectOption('#country', valueMap[country.toLowerCase()]);
});
```

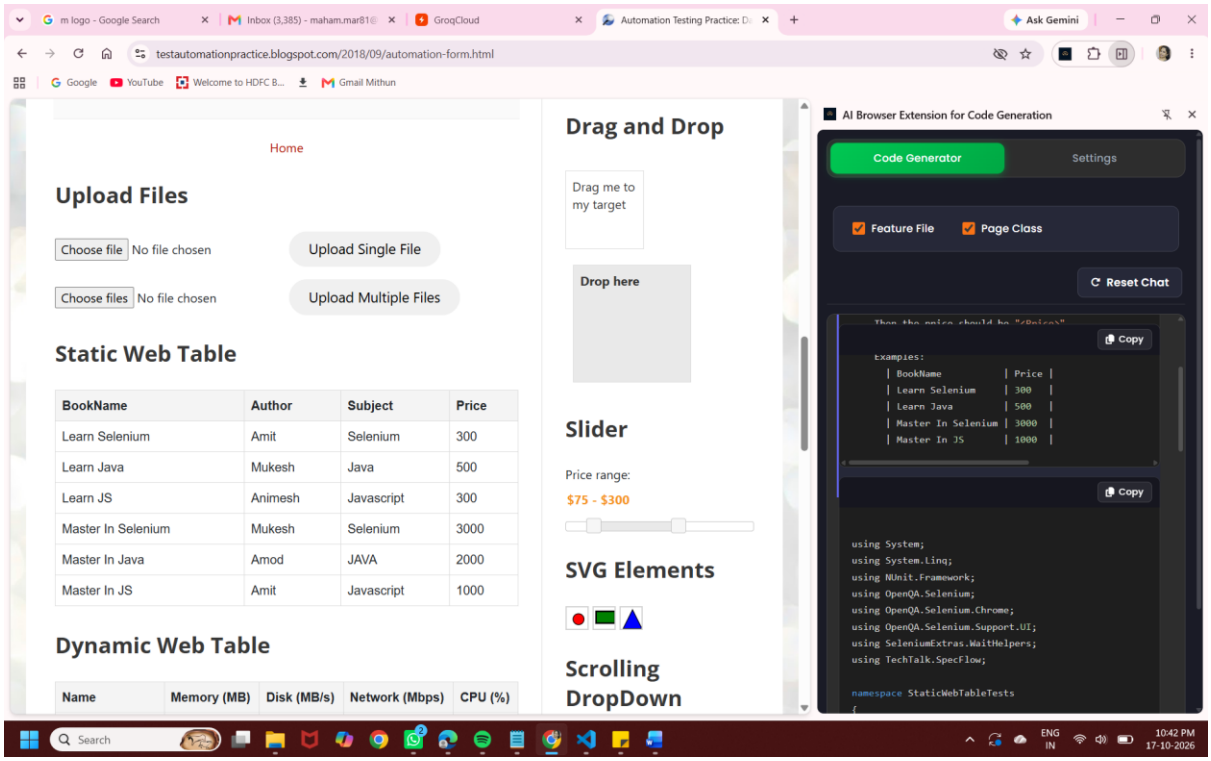
- **Selenium with C#**






Good. Then approve the implementation in Agent
Completed 13 steps in 56s +82 -2
Implemented C# + Selenium support in chat.js:610 and prompts.js:48. C# now routes to dedicated page-object and combined Cucumber/Reqnroll step-definition prompts, including the no-selection fallback. Feature-only generation and existing Java/Selenium and TypeScript/Playwright routing are preserved; no UI changes were needed.
Validation passed: both scripts pass node --check, the new prompts resolve and substitute DOM/URL values in a runtime smoke test, and editor diagnostics report no errors.
10:36 PM • GPT-6 Luna • 1.8 credits
chat.js
Describe what to build
Agent
Auto
Balance
Local
Default permissions


AI Browser Extension for Code Generation
Code Generator
Settings
Supported Language
C#
Browser Engine
Selenium
Select LLM Provider
Groq
Select Model
openai/gpt-oss-120b
Groq API Key




```
Feature: Verify book prices in static web table
  As a tester
  I want to verify that each book listed in the static web table displays the correct price
  So that the data presented to users is accurate

  Scenario Outline: Verify price for a specific book
    Given I open the automation practice page
    When I look for the book "<BookName>"
    Then the price should be "<Price>"

    Examples:
      | BookName           | Price |
      | Learn Selenium     | 300   |
      | Learn Java         | 500   |
      | Master In Selenium | 3000  |
      | Master In JS       | 1000  |
```

```
using System;
using System.Linq;
using NUnit.Framework;
using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;
using OpenQA.Selenium.Support.UI;
using SeleniumExtras.WaitHelpers;
using TechTalk.SpecFlow;

namespace StaticWebTableTests
{
    [Binding]
    public class BookPriceSteps
    {
        private IWebDriver _driver;
        private WebDriverWait _wait;
        private string _bookName;

        private const string Url = "https://testautomationpractice.blogspot.com/2018/09/automation-form.html";

        [BeforeScenario]
        public void BeforeScenario()
        {
            var options = new ChromeOptions();
            options.AddArgument("--headless"); // optional: run headless
            _driver = new ChromeDriver(options);
            _driver.Manage().Window.Maximize();
            _driver.Navigate().GoToUrl(Url);
            _wait = new WebDriverWait(_driver, TimeSpan.FromSeconds(10));
        }

        [AfterScenario]
        public void AfterScenario()
        {
            _driver.Quit();
        }

        [Given(@"I open the automation practice page")]
        public void GivenIOpenTheAutomationPracticePage()
        {
            // Ensure the table is present before proceeding
            _wait.Until(ExpectedConditions.ElementExists(By.CssSelector("table[name='BookTable']")));
        }

        [When(@"I look for the book ""(.*)""")]
        public void WhenILookForTheBook(string bookName)
        {
            _bookName = bookName;
        }

        [Then(@"the price should be ""(.*)""")]
        public void ThenThePriceShouldBe(string expectedPrice)
        {
            // Locate the table
            var table = _wait.Until(
                ExpectedConditions.ElementExists(By.CssSelector("table[name='BookTable']"))
            );

            // Get all rows (including header)
            var rows = table.FindElements(By.CssSelector("tbody tr"));

            IWebElement targetRow = null;

            // Skip the header row (first row) and search for the book name
            foreach (var row in rows.Skip(1))
            {
```